

IV. ÉMULATION D'UN SYSTEME LINUX GRACE A JSLINUX

👉 Source : lyceum.fr
Pour manipuler sous Linux, on utilisera l'émulateur de système mis au point par Fabrice Bellard accessible à l'adresse :

<https://bellard.org/jslinux/vm.html?url=https://bellard.org/jslinux/buildroot-x86.cfg>

1. LA MANUEL D'UTILISATION

La commande `man` (pour `manual`) permet d'obtenir la page du manuel d'utilisation d'une commande. S'avère utile pour connaître la syntaxe exacte de la commande et les options possible

⚠ Commande non implémentée sur le simulateur

2. NOTION D'UTILISATEUR ET DE MACHINE

Qui suis-je (renvoie le nom de l'utilisateur) : `whoami`

Où suis-je (renvoie le dossier (*directory*) courant) ? `pwd`

Par défaut quand on ouvre un terminal on se trouve dans son dossier `$HOME`:

- `/home/username` pour l'utilisateur `username`
- `/root/` si vous êtes `root`, l'utilisateur administrateur du système.

Exo 2. <https://bellard.org/jslinux/vm.html?url=https://bellard.org/jslinux/buildroot-x86.cfg>

👉 Ouvrez une console, vous devriez avoir quelque chose qui ressemble à cela :

```
Loading...

Welcome to JS/Linux (x86)

Use 'vflogin username' to connect to your account.
You can create a new account at https://vfsync.org/signup .
Use 'export_file filename' to export a file to your computer.
Imported files are written to the home directory.

[root@localhost ~]#
```

ci-dessus la console de l'utilisateur `root` qui utilise un ordinateur qui se nomme `localhost` (`root@localhost`).

Les commandes sont saisies après le signe `#`.

a. Essayer les commandes suivantes:

- `whoami`
- `hostname`

b. Créer votre login : « You can create a new account at <https://vfsync.org/signup> . »

c. La commande "`su`" (Switch User, appelée plus communément (à tort semble-t-il) "Super Utilisateur") permet d'ouvrir une session avec l'ID (IDentifiant) d'un autre utilisateur, ou de démarrer un nouveau shell de connexion.

```
su toto
```

d. Après avoir entré votre mot de passe, vous êtes connecté en tant qu'utilisateur "`toto`", vous êtes toujours dans le même répertoire de travail que précédemment et vous bénéficiez des réglages par défaut de l'utilisateur précédent (shell de connexion, variables d'environnement, permissions diverses...)

e. Pour revenir à l'utilisateur `root` : `exit`

3. SYSTÈME DE FICHIERS LINUX

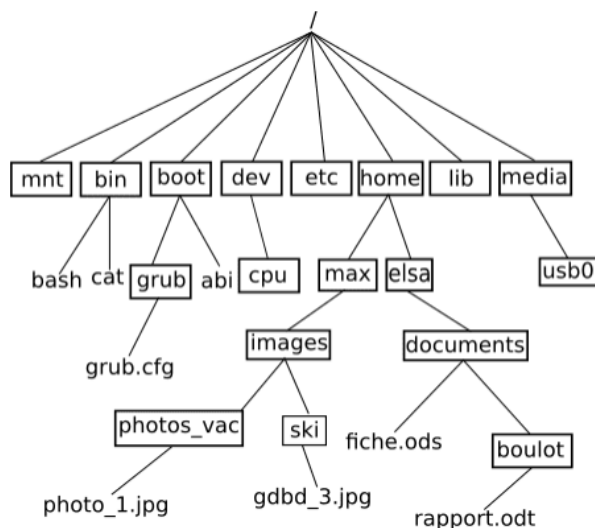
Sous Linux : pas d'unités logiques (C :,D :,...) comme sous Windows

Une seule arborescence dont la racine est désignée par le symbole /

Tous les volumes de données (Disque dur, lecteur USB,...) figurent dans cette arborescence

/	racine de l'arborescence
/bin	commandes binaires utilisateur essentielles (pour tous les utilisateurs)
/boot	fichiers statiques du chargeur de lancement
/dev	fichiers de périphériques
/etc	configuration système spécifique à la machine
/home	répertoires personnels des utilisateurs
/lib	bibliothèques partagées essentielles et modules du noyau
/mnt	point de montage pour les systèmes de fichiers montés temporairement
/media	point de montage des clés USB, CD-ROM
/proc	système de fichiers virtuel d'information du noyau et des processus
/root	répertoire personnel de root
/sbin	binaires système (binaires auparavant mis dans /etc)
/tmp	fichiers temporaires

Ex 4 :



Dans le schéma ci-dessus on trouve des répertoires (noms entourés d'un rectangle, exemple : home) et des fichiers (uniquement des noms `grub.cfg`). À noter : les extensions des noms de fichiers, par exemple le `cfg` de `grub.cfg`, ne sont pas obligatoires dans les systèmes de type UNIX, par exemple, `bash` est bien un nom de fichier et il n'a pas d'extension.

👉 Pour indiquer la position d'un fichier (ou d'un répertoire) dans l'arborescence, chemin absolu ou chemin relatif ?

Le chemin absolu doit indiquer le chemin depuis la racine.

Ex 5 : le chemin absolu du fichier `fiche.ods` sera : `/home/elsa/documents/fiche.ods`

⚠ le chemin démarre bien de la racine / (attention les symboles de séparation sont aussi des /)

Le chemin relatif indique le chemin depuis le répertoire courant.

Ex 6 : Le chemin relatif permettant d'accéder au fichier `photo_1.jpg` depuis le répertoire `max` est :
`images/photo_vac/photo_1.jpg`



L'absence du « / » au début du chemin indique que le départ est le répertoire courant.
On peut aussi expliciter cet indication en utilisant « . / » au début du chemin.

Ex 7 : Pour indiquer le chemin relatif pour accéder au fichier `gdbd_3.jpg` depuis le répertoire `photos_vac` :



Il faut remonter d'un niveau dans l'arborescence pour se retrouver dans le répertoire `images` et ainsi pouvoir repartir vers la bonne branche. Pour ce faire il faut utiliser 2 points : ..
`../ski/gdbd_3.jpg`

Rq1 On peut remonter de plusieurs crans : `../../` depuis le répertoire `photos_vac` permet de remonter dans le répertoire `max`

Exo 3. En vous basant sur l'arborescence ci-dessus,

1. déterminer le chemin absolu permettant d'accéder au fichier :

- `cat`
- `rapport.odt`

2. Toujours en vous basant sur l'arborescence ci-dessus, déterminer le chemin relatif permettant d'accéder au fichier :

- `rapport.odt` depuis le répertoire `elsa`
- `fiche.ods` depuis le répertoire `boulot`



Attention à la casse : les systèmes de type `UNIX` sont sensibles à la casse (il différencie les caractères majuscules et les caractères minuscules) : le répertoire `Documents` et le répertoire `documents` sont 2 répertoires différents.

a) Quelques commandes usuelles

La commande `cd` (pour *change directory*) permet de changer le répertoire courant. Il suffit d'indiquer le chemin (relatif ou absolu) qui permet d'atteindre le nouveau répertoire :

La commande `ls` permet de **lister** les fichiers et répertoires du dossier courant

Ex 8 :

- `ls /chemin/vers/dossier`: liste les fichiers et répertoires du dossier donné en argument/
- `cd /root`

Ex 9 : En utilisant l'arborescence ci-dessus :

- si le répertoire courant est le répertoire `elsa` et que vous voulez vous rendre dans le répertoire `documents`, il faudra saisir la commande : `cd documents` (relatif) ou `cd /home/elsa/documents` (absolu)
- si le répertoire courant est le répertoire `photos_vac` et que vous voulez vous rendre dans le répertoire `ski`, il faudra saisir la commande : `cd ../ski` (relatif) ou `cd /home/max/images/ski` (absolu)
- si le répertoire courant est le répertoire `boulot` et que vous voulez vous rendre dans le répertoire `documents`, il faudra saisir la commande : `cd ..` (relatif) ou `cd /home/elsa/documents` (absolu)

Rq2

3. visualiser le contenu d'un fichier : `cat nom_du_fichier`
4. Créer un dossier : `mkdir chemin vers le/dossier`

Exo 4. Après avoir ouvert une console, utilisez la commande `ls` depuis votre répertoire personnel.

La commande `mkdir` permet de créer un répertoire dans le répertoire courant. La commande est de la forme `mkdir nom_du_répertoire`

Rq3. Remarque : il est préférable de ne pas utiliser de caractères accentués dans les noms de répertoire (ou de fichier). Il en est de même pour les espaces (à remplacer par des caractères tirets bas `_`)

Exo 5. Après avoir ouvert une console, utilisez la commande `mkdir` afin de créer un répertoire `test_nsi` dans votre répertoire personnel.

La commande `rm` permet de supprimer un fichier ou un répertoire. La commande est de la forme `rm nom_du_répertoire_ou_nom_du_fichier`,
pour **supprimer un répertoire non vide**, utiliser `rm -rf ...`

Rq4. La plupart des commandes UNIX peuvent être utilisées avec une ou des options. Par exemple, pour supprimer un répertoire non vide, il est nécessaire d'utiliser la commande `rm` avec l'option `-r` : `rm -r nom_du_répertoire`

La commande `touch` permet de créer un fichier vide. La commande est de la forme `touch nom_du_fichier_à_créer`

La commande `cp` permet de copier un fichier. La commande est de la forme `cp /répertoire_source/nom_fichier_à_copier /répertoire_destination/nom_fichier`

Exo 6. Placez-vous dans le répertoire `test_nsi` créé précédemment.

Créez un fichier `test.txt`.

Créez un répertoire `doc`.

Copiez le fichier `test.txt` dans le répertoire `doc`.

Effacez le répertoire `doc` (et son contenu).

b) Droits des utilisateurs et groupes

L'option `-l` (pour *long*) passée à la commande `ls` permet de connaître les **droits** sur les fichiers



La commande `ls -l` précise dans sa sortie les attributs suivants de chaque fichier :

permission type utilisateur propriétaire groupe propriétaire taille date nom

```
ls -l Documents/git/2heoinfo/flask-csv/

total 28
-rw-r--r-- 1 ben docker 2690  4 juil. 10:39 brouillon.ipynb
-rw-r--r-- 1 ben docker  120  4 juil. 12:30 data.csv
-rw-r--r-- 1 ben docker  881  4 juil. 13:53 flask-csv.py
-rw-r--r-- 1 ben docker 1064  4 juil. 13:42 LICENSE
drwxr-xr-x 2 ben docker 4096  2 juil. 22:26 pages
-rw-r--r-- 1 ben docker  501  4 juil. 12:39 README.md
-rw-r--r-- 1 ben docker   70  4 juil. 10:07 run-server.sh
```

Les droits sont sous la forme `drwxrwxrwx`.

- `d` indique qu'il s'agit d'un dossier
- `r`: droit de lecture
- `w`: droit d'écriture
- `x`: droit d'exécution

👉 Les droits sont attribués dans l'ordre à l'utilisateur (*user*), au groupe (*group*), aux autres (*others*).

Ex 10 : `-rw-r--r--`: fichier avec droit de lecture et d'écriture pour l'utilisateur propriétaire, mais uniquement des droits de lecture pour les membres du groupe propriétaire et les autres utilisateurs.

Ex 11 : `drwxr-xr-x`: dossier avec droit de lecture, écriture et exécution pour l'utilisateur propriétaire, mais uniquement des droits de lecture et d'exécution pour les membres du groupe propriétaire et les autres utilisateurs.

La commande `chmod` (*change mode*) permet de gérer les droits des fichiers et répertoires

👉 Un utilisateur a le droit de faire un `chmod` sur un fichier :

- s'il est `root` ;
- ou s'il est le propriétaire du fichier en question.

👉 Les options passées à la commande `chmod` sont indiquées de la manière suivante :
`chmod options modes fichiers`

Pour un fichier : `chmod [u g o a] [+ - =] [r w x] nom_du_fichier`

👉 Pour le contenu d'un répertoire (de façon *récursive*) :

`chmod -R [u g o a] [+ - =] [r w x] nom_du_répertoire`

Les lettres `u g o a` désignent les types de propriétaires concernés :

- `u` propriétaire (*user*)
- `g` groupe (*group*)
- `o` les autres (*others*)
- `a` tous (*all*)

👉 Les lettres `+ - =` désignent les changements d'état: `+` et `-` pour ajouter ou retirer un type de droit aux droits courants, et l'opérateur `=` pour les écraser.
Les lettres `r w x` désignent les permissions : *read*, *write*, *execute*.

Ex 12 : Dans le cas précédent si l'on souhaite que tous les utilisateurs du groupe `docker` puissent lire et écrire dans le fichier `brouillon.ipynb`, on lance la commande `chmod g+w` sur le fichier:

`chmod g+w Documents/git/2heoinfo/flask-csv/brouillon.ipynb`

👉 On peut alors vérifier le changement de droits avec `ls -l`

```
ben@coolerarch:~$ ls -l Documents/git/2heoinfo/flask-csv/
total 28
-rw-rw-r-- 1 ben docker 2690  4 juil. 10:39 brouillon.ipynb
...
```

La commande `chown` (*change owner*) permet de gérer les propriétaires des fichiers et répertoires

👉 L'utilisateur racine(`root`) peut changer le propriétaire d'un fichier ou d'un dossier
La syntaxe générale de la commande est :

`chown [-hLPR] [utilisateur][:groupe] cible1 [cible2 ..]`

- Rendre l'utilisateur `john` propriétaire du fichier `toto.txt`: `chown john /home/jane/toto.txt`

- Pour modifier le propriétaire de tout le contenu d'un dossier : `chown -Rf utilisateur1 /home/dossier1`
- Pour modifier l'utilisateur et le groupe : `chown ben:prof /home/jane/toto.txt`

Exo 7.

- a. Dans la capture ci-dessous, analyser les droits utilisateurs du fichier photo.jpg

```
david@david-XPS-13-9343 ~/nsi $ ls -l
total 4
-rw-r--r-- 1 david david  0 avril 13 19:58 fic.txt
drwxr-xr-x 2 david david 4096 avril 13 20:05 info
-rw-r--r-- 1 david david  0 avril 12 09:19 photo.jpg
david@david-XPS-13-9343 ~/nsi $
```

- b. Analysez attentivement l'enchaînement de commandes suivantes :

```
david@PC-Bureau ~ $ cd nsi
david@PC-Bureau ~/nsi $ ls
david@PC-Bureau ~/nsi $ touch toto.txt
david@PC-Bureau ~/nsi $ ls -l
total 0
-rw-rw-r-- 1 david david 0 avril 21 10:20 toto.txt
david@PC-Bureau ~/nsi $ chmod o+x toto.txt
david@PC-Bureau ~/nsi $ ls -l
total 0
-rw-rw-r-x 1 david david 0 avril 21 10:20 toto.txt
david@PC-Bureau ~/nsi $ chmod g-w toto.txt
david@PC-Bureau ~/nsi $ ls -l
total 0
-rw-r--r-x 1 david david 0 avril 21 10:20 toto.txt
david@PC-Bureau ~/nsi $
```

- c. Créez un répertoire `test_nsi2` dans votre répertoire personnel. Placez-vous dans le répertoire `test_nsi2`. Créez un fichier `titi.txt`, vérifiez les permissions associées à ce fichier. Modifiez les permissions associées au fichier `titi.txt` afin que les autres utilisateurs aient la permission écriture

V. Exos

Pour une synthèse des commandes , voir <https://juliend.github.io/linux-cheatsheet/>

Exo 8. On suppose que le répertoire personnel de l'utilisateur courant est vide.

1. Décrire l'effet de chacune des commandes successives suivantes

```
- cd ~
- mkdir NSI
- mkdir NSI/TP_SHELL
- cd NSI/TP_SHELL
- cd ..
- ls
- chmod u+rw,g-rwx,o-rwx TP_SHELL
```

2. Ouvrir un terminal et vérifier que tout se passe comme prévu.

Exo 9. Motifs glob : source http://wiki.linux-france.org/wiki/Les_commandes_fondamentales_de_Linux

Nous pouvons demander à `ls` de lister uniquement les fichiers dont les noms contiennent des caractères donnés. Gagnons le répertoire `/bin`: `cd /bin` puis demandons par exemple la liste des seuls noms de fichiers dont les noms commencent par la lettre "g": `ls g*`

L'argument `"g*"` que nous venons de fournir est un motif, auquel on peut penser comme à une sorte de filet dont les mailles sont définies par nous (utilisateur), en respectant un certain nombre de conventions, et qui est ensuite appliqué (jeté) par le shell afin de sélectionner des noms de fichiers qui y correspondent.

Les principales conventions valides dans ce contexte, donc lorsque l'on définit un motif de sélection de noms de fichiers destiné au shell, sont:

l'étoile (ou plutôt l'astérisque) est un caractère générique sélectionnant **n'importe quelle séquence de caractères** (potentiellement vide)

le point d'interrogation sélectionnant **n'importe quel caractère**.

[c1-cn] sélectionne tous les caractères contigus de la plage de c1 à cn.

1. Pour chacun des motifs *glob* ci-dessous donner une suite de caractères (de longueur ≥ 1) reconnue par le motif :
 - *.jpg
 - *.???
 - ?????
 - ?*?
 - ???0.???*
 - [0-9]*[a-z]
 - [A-Z]??.jpg
2. Tester les réponses dans un terminal. Pour cela, se placer dans le répertoire TP_SHELL créé lors de l'exercice précédent, créer les fichiers vides au moyen de la commande « touch *nom du fichier* » et tester l'expression au moyen de « ls *motif* ». Le fichier créé avec touch doit être listé (au moins lui).

Par ex. :

```
[elsa@localhost NSI_TEST]$ ls
[elsa@localhost NSI_TEST]$ mkdir TP_SHELL
[elsa@localhost NSI_TEST]$ cd TP_SHELL
[elsa@localhost TP_SHELL]$ touch unfichier.txt
[elsa@localhost TP_SHELL]$ ls *txt
unfichier.txt
```

Exo 10. On suppose que l'on se trouve dans un répertoire TEST, vide, et que l'on exécute les commandes successives suivantes :

- mkdir a b c d
- touch a/t.txt d/foo.txt
- cd c
- mkdir ../b/e f g
- cd ..
- cp */*.txt c/g
- rm -rf d

Dessiner l'arborescence, à partir du répertoire TEST, des fichiers et répertoires qui résultent de cette succession de commandes